

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2 Learning Outcomes

Students will be able to:

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3 Background Theory

A Convolutional Neural Network consists of:

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is:

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where:

X : Input Image

K : Kernel (Filter)

Y : Feature Map

The output feature map size is:

$$\frac{N - F + 2P}{S} + 1$$

where:

N : Input Size

F : Filter Size

P : Padding

S : Stride

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

4 Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

5 Numerical Example 1: Convolution

Consider the following input image:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel:

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output:

First position: $1(1) + 2(0) + 4(0) + 5(1) = 6$

Second position: $2(1) + 3(0) + 5(0) + 6(1) = 8$

Third position: $4(1) + 5(0) + 7(0) + 8(1) = 12$

Fourth position: $5(1) + 6(0) + 8(0) + 9(1) = 14$

Feature Map:

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

6 Numerical Example 2: Max Pooling

Feature map:

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter:

$$\text{Window 1: } \begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

$$\text{Window 2: } \begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

$$\text{Window 3: } \begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

$$\text{Window 4: } \begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output:

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

7 Numerical Example 3: Parameter Calculation

Suppose:

- **Input Image:** $32 \times 32 \times 3$
- **Convolution Layer:** Filters = 16, Kernel = 3×3

Parameters:

$$(3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 448$$

Therefore,

$$\text{Total trainable parameters} = 448.$$

8 Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Classes: 10
- Image Size: $32 \times 32 \times 3$

9 Experimental Procedure

- **Task 1: Dataset Exploration**

- Load CIFAR-10.
- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

- **Task 2: Convolution Layer**

- Implement a convolution layer.
- Compare kernel sizes 3×3 , 5×5 , and 7×7 .
- Record feature map sizes.

- **Task 3: Hyperparameter Study**

- Compare stride = 1 and stride = 2.
- Compare padding = Same and padding = Valid.
- Compute output dimensions.

- **Task 4: Feature Map Visualization**

- Visualize feature maps after the first convolution layer.
- Display at least eight feature maps.

- **Task 5: Pooling Comparison**

- Compare Max Pooling.
- Compare Average Pooling.
- Compare output size and accuracy.

- **Task 6: CNN Construction**

- Construct the following CNN:

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

Training Configuration

- Optimizer: Adam
- Epochs: 20
- Batch Size: 32

- **Task 7: Model Evaluation**

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

10 Source Code

Source: Github

Github Repository Link:

https://github.com/ssvibitha/DeepLearningLab_2026-27/blob/main/Lab3_CNN/DL_Lab3.ipynb

11 Plots and Inference

1. Sample Images

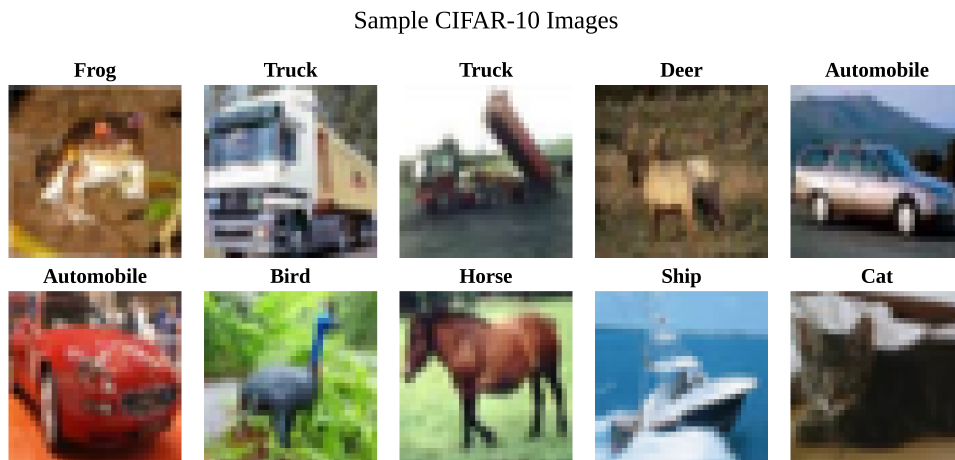


Figure 1: Sample Images

Inference:

The figures show sample images from the CIFAR-10 dataset. The images belong to 10 different categories such as Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck. Each image is 32×32 pixels and contains three color channels (RGB).

2. Class Distribution



Figure 2: Class Distribution

Inference:

The histogram shows the class distribution of the CIFAR-10 dataset. The dataset consists of images across 10 different classes such as Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck. In the training set, each class contains 5000 images each. Overall the training set contains 50000 images spanning across 10 different classes.

3. Training and Validation Accuracy

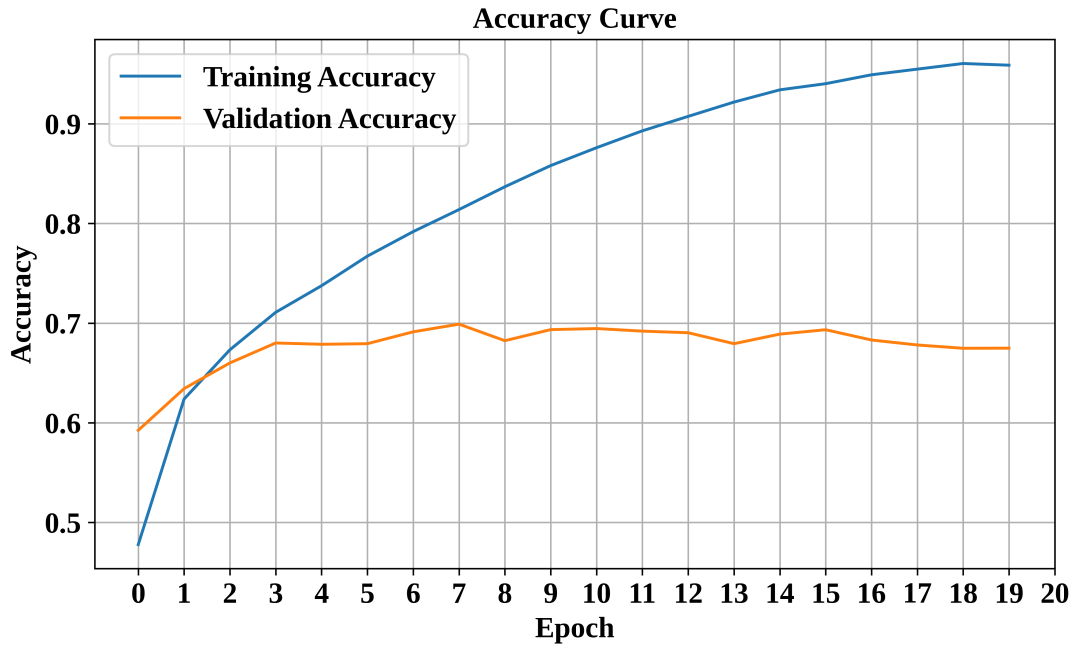


Figure 3: Accuracy Curve

Inference:

The plot shows the training and validation accuracy curves for 20 epochs. The training accuracy steadily increases from around 50 percent to over 95 percent as the number of epochs increases. Validation accuracy improves from 60 percent and reaches around 70 percent in the 7th epoch. Then it remains stable for the rest of the epochs. A noticeable gap develops between training and validation accuracy after a few epochs. This indicates that the model is overfitting the training data. Further training improves training accuracy but does not significantly improve validation performance.

4. Training and Validation Loss

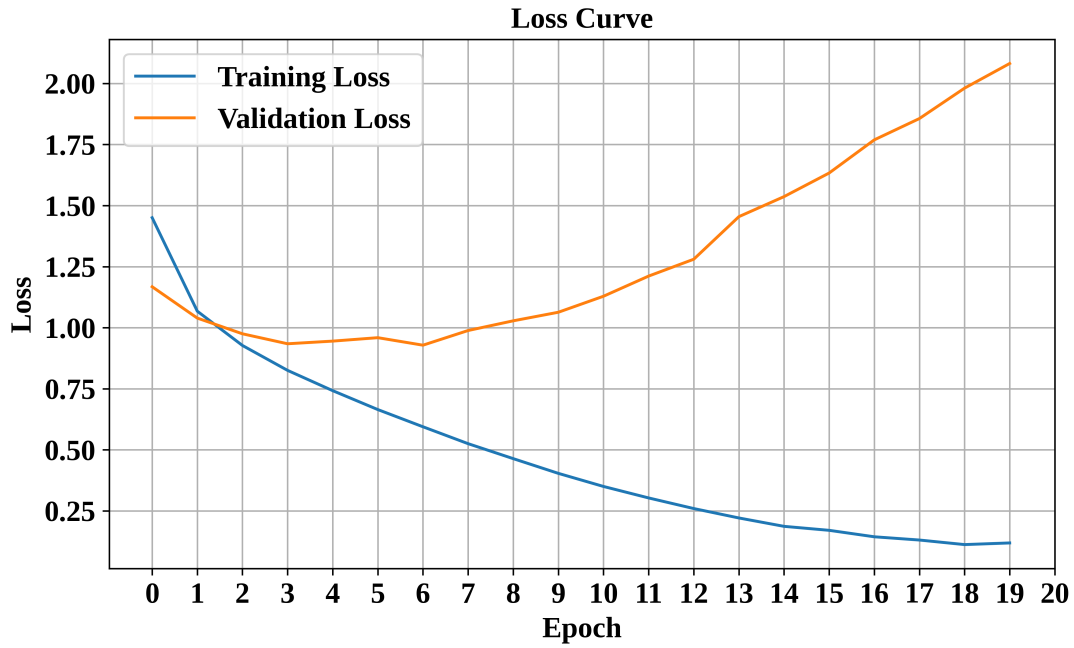


Figure 4: LossCurve

Inference:

The plot shows the training and validation loss for 20 epochs. Training loss continuously decreases throughout the epochs. This indicates that the model learns the training data effectively. Validation loss decreases initially and reaches its minimum around 4–6 epochs. Afterwards validation loss steadily increases while training loss continues to decrease. The increasing gap between the accuracy and validation loss curves shows that the model is overfitting.

5. Feature Maps

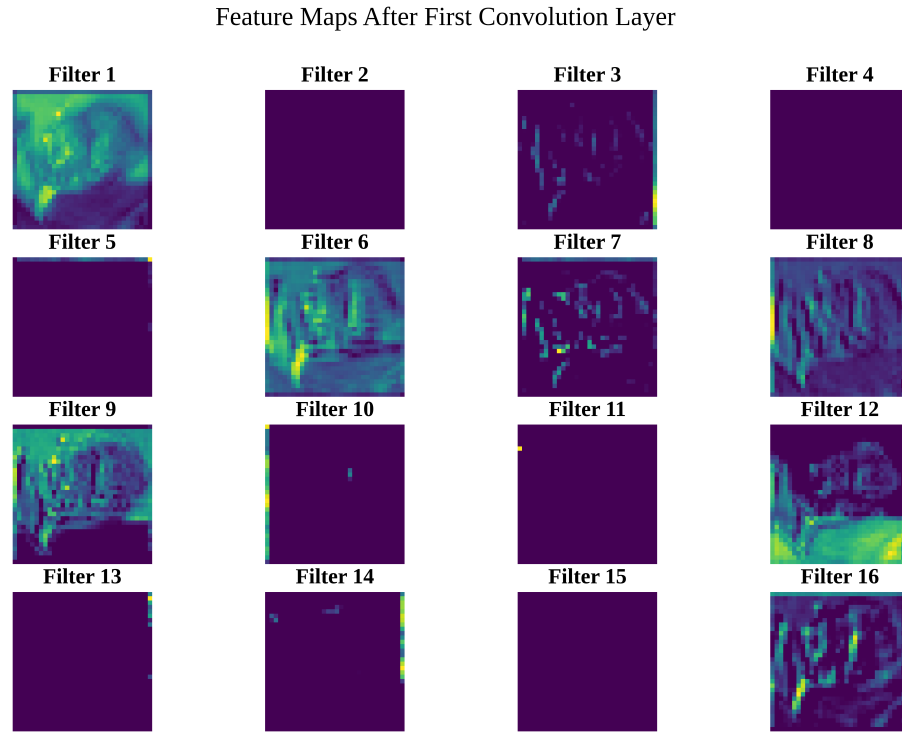


Figure 5: Feature Maps

Inference:

The plot shows what each of the 16 filters learns after the first convolutional layer. The feature maps show that different filters detect various aspects of the image, such as edges, textures, contours, and color patterns. Some filters produce strong activations, while others show weaker or minimal responses. This indicates that the first convolutional layer effectively extracts features from the input images.

6. Confusion Matrix

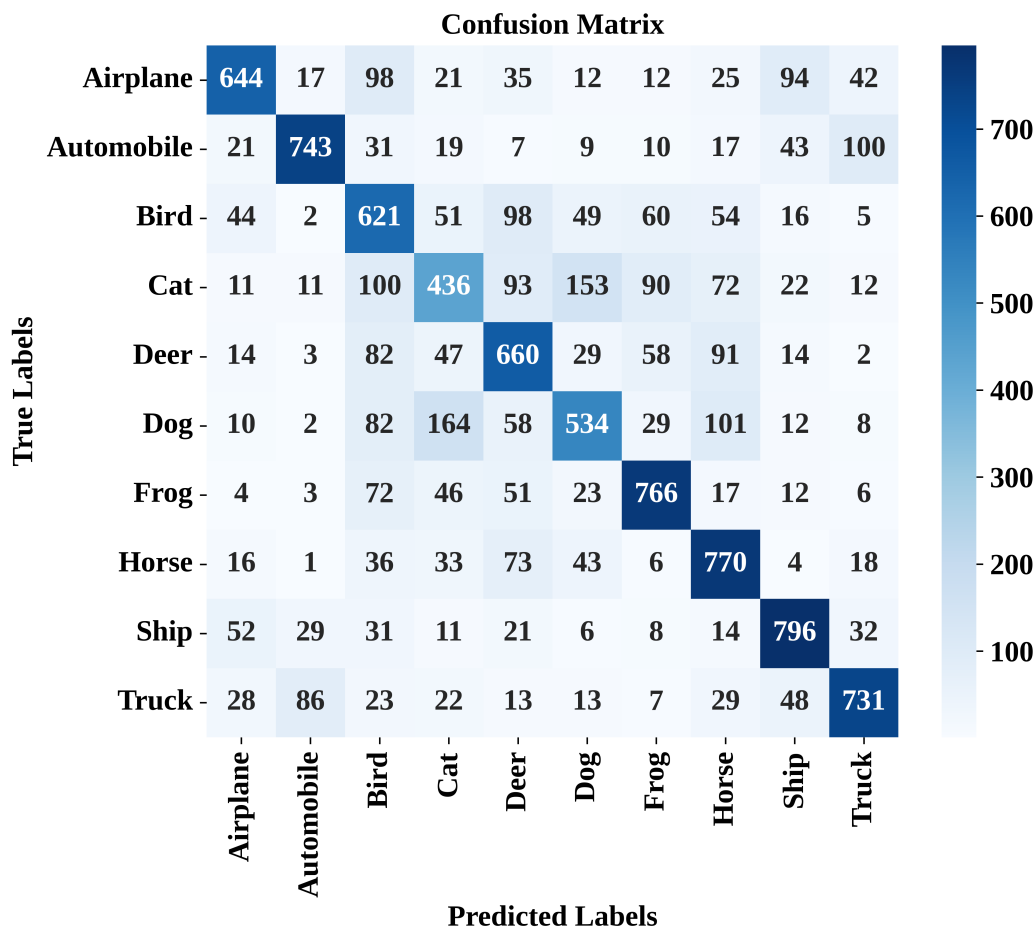


Figure 6: Confusion Matrix

Inference:

The figure shows the confusion matrix of the model's predictions. The model performs relatively well for classes such as Ship, Frog, Horse, and Truck, which have a high number of correct predictions. From the figure, 100 Automobile images were misclassified as Truck, while 164 Dog images were misclassified as Cat. Additionally, 86 Truck images were misclassified as Automobile. Overall, the confusion matrix shows that the model can distinguish most classes but struggles with visually similar classes such as Dog and Cat, Automobile and Truck.

12 Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.
2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.
3. Compare ReLU and Sigmoid activation functions.
4. Replace Max Pooling with Average Pooling and compare the results.
5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

13 Results

Metric	Value
Training Accuracy	95.88%
Testing Accuracy	67.01%
Precision	67.34%
Recall	67.01%
F1-score	66.92%
Number of Parameters	545,098

Table 1: Model Performance Results

14 Discussion

Answer the following:

1. **Why is convolution preferred over fully connected layers for images?**

Convolution preserves the spatial structure of an image and detects local features such as edges, textures, contours, and patterns. It uses shared weights, so the same filter can detect the same feature across different parts of the image, providing translation invariance. This makes convolutional layers more efficient and require fewer parameters than fully connected layer

2. **How does stride affect the feature map size?**

Stride determines how many pixels the filter moves at each step while scanning an image. Smaller stride means the filter moves in smaller steps and examines more regions producing a larger feature map with more detailed information. A larger stride means the filter moves

farther at each step so fewer regions are examined, resulting in a smaller feature map and lower computational cost.

3. What is the role of padding?

Padding adds extra pixels around the edges of an image before applying convolution. It allows the filter to cover the border and corner pixels properly, which might otherwise be used fewer times than the pixels in the center. Padding also helps preserve the spatial dimensions of the feature map and allows the network to retain more information from the edges of the image.

4. Why is pooling used?

Pooling is used to reduce the spatial dimensions of feature maps while keeping the most important information. For example, max pooling selects the maximum value from a small region, which helps preserve strong features such as edges and textures. This reduces the number of computations and parameters in subsequent layers making the CNN faster and less prone to overfitting. Pooling also makes the model less sensitive to small shifts or changes in the position of features in an image.

5. How do feature maps represent image characteristics?

Feature maps show the features detected by convolutional filters at different locations in an image. Early layers usually detect simple features like edges and textures, while deeper layers detect more complex patterns.

6. Why do CNNs require fewer parameters than MLPs?

CNNs use local connections and shared filter weights instead of connecting every neuron to every pixel. A convolutional filter only looks at a small region of the image at a time instead of the entire image. The same filter is reused across different locations, so the same set of weights can detect a feature such as an edge anywhere in the image. Therefore, CNNs need far fewer trainable parameters than MLPs, where each neuron is connected to every input pixel with separate weights.

15 References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.